

Analysis of Crime Data in Los Angeles from 2020 to 2024

Ayoola Ogunbona

13 March 2025

Introduction

Crime analysis is vital in shaping public safety policies, optimizing law enforcement resource allocation, and improving urban planning. This study analyzes crime trends in Los Angeles from 2020 to 2024, utilizing data obtained from the LA Crime Open Data Portal. The primary goal is to uncover patterns related to the timing of crimes, their geographic distribution, and the most commonly reported offenses. Through exploratory data analysis (EDA), this project aims to identify high-crime areas, seasonal trends, and demographic factors influencing crime occurrences.

This study seeks to answer three key questions:

1. When do most crimes occur? (Analyzing crime timing patterns by time of day, month, and year)
2. Where do crimes happen most frequently? (Conducting geospatial analysis of crime locations)
3. What are the most common types of crimes? (Identifying major crime categories and trends over time)

By exploring these aspects, better understanding of crime hotspots, high-risk periods, and trends that could assist law enforcement agencies and city officials in implementing proactive crime prevention strategies.



Data Explained

The dataset was obtained from the [LA Crime Open Data Portal](#). According to data from data.gov **(Update as of January 18, 2024)**, the LAPD is currently facing challenges in posting crime data and is working diligently to resolve these issues. As a result, they are updating the data every **two weeks instead of weekly**, which has led to 2024 data being less comprehensive than in previous years. Thank you for your understanding. It contains over **1,000,000 records** of reported crimes in Los Angeles from **2020 to 2024**. It includes essential information such as the type of crime, location, victim demographics, and time of occurrence.

The steps for data cleansing for this analysis included:

- Removed missing values in critical columns (e.g., crime type, date, location).
- Converted date and time columns to proper datetime formats.
- Filtered out invalid victim ages (e.g., 0 or negative values).
- Standardized categorical values (e.g., gender, ethnicity).

Seasons were defined as follows:

- Winter: January, February, and December
- Spring: March, April, and May
- Summer: June, July, and August
- Fall: September, October, and November

Hypotheses:

- Crime rates increase during summer months due to higher outdoor activity.
- Certain areas in LA consistently experience higher crime rates compared to others.
- Weapons are more frequently used in specific crime types.
- Nighttime crime rates are higher due to reduced law enforcement presence.

Data Dictionary:

Measure Name	Description	Datatype
Case_Number	The Official file number	int64
Date_Reported	Date of crime report	object
Date_Occurred	Date of crime occurrence(YYYY-MM-DD)	datetime64[ns]
Time_Occurred	Time of crime occurrence	int64
Area_Name	The 21 Geographic Areas or Patrol Divisions .	object
Rpt Dist No	Code representing a sub-area within a Geographic Area.	int64
Crime_Code	Code representing the type of crime committed	int64
Crime_Description	What kind of crime was committed	object

Measure Name	Description	Datatype
Victim_Age	Indicates the age of the victim.	int64
Victim_Sex	F: Female M: Male	object
Victim_Ethnicity	The race description of Victim	object
Premises_Code	The code used for that premises	float64
Premises_Description	The description of the area	object
Weapon_Description	Defines the Weapon Used Code provided.	object
Case_Status	The Status of the case	object
Case_Status_Description	The description of the case status	object
Crm Cd 1	Indicates the crime committed.	float64
LOCATION	Street address of crime incident .	object
Latitude	Latitude Coordinate	float64
Longitude	Longitude Coordinate	float64
Year	The year of occurrence	int64
Month	The month of occurrence	Object
Weapon_Used	Checking if a weapon was used or not	int64
Season	The season of crime occurrence: Winter, Spring, Fall, Summer	Object

1004847 non-null float64

Results

```
In [7]: #Load up modules
import pandas as pd
import numpy as np

#Load up visuals
import seaborn as sns
import matplotlib.pyplot as plt
import plotly.express as px

#set up notebook to display multiple output in one cell
from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = "all"
```

```
In [8]: #read in final files
df = pd.read_csv("Cleaned_Crime_Data_Final.csv")

#show file
df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 724907 entries, 0 to 724906
Data columns (total 25 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Case_Number                          724907 non-null  int64
1   Date_Reported                       724907 non-null  object
2   Date_Occurred                       724907 non-null  object
3   Time_Occurred                       724907 non-null  object
4   Area_Name                           724907 non-null  object
5   Rpt Dist No                         724907 non-null  int64
6   Crime_Code                          724907 non-null  int64
7   Crime_Description                   724907 non-null  object
8   Victim_Age                         724907 non-null  int64
9   Victim_Sex                         724907 non-null  object
10  Victim_Ethnicity                   724907 non-null  object
11  Premises_Code                     724907 non-null  float64
12  Premises_Description              724907 non-null  object
13  Weapon_Description                724907 non-null  object
14  Case_Status                      724907 non-null  object
15  Case_Status_Description           724907 non-null  object
16  Crm Cd 1                         724907 non-null  float64
17  LOCATION                         113185 non-null  object
18  Latitude                         724907 non-null  float64
19  Longitude                        724907 non-null  float64
20  Year                             724907 non-null  int64
21  Month                            724907 non-null  object
22  Weapon_Used                      724907 non-null  int64
23  month                            724907 non-null  int64
24  season                           724907 non-null  object
dtypes: float64(4), int64(7), object(14)
memory usage: 138.3+ MB

```

Geospatial Crime Hotspots and Top 10 Crime Areas in Los Angeles

To establish a geographic context, we begin by **mapping crime locations** across Los Angeles. This visualization helps identify high-crime areas, which is essential for law enforcement resource allocation and urban safety planning.

Observation:

- The **highest concentrations** of crime reports are seen in **Central LA, 77th Street, and Pacific**, aligning with the expectation that **densely populated areas** experience more crime.
- This supports the hypothesis that **certain areas in LA have persistently high crime rates**.
- Particularly in locations with commercial activity, nightlife, and high residential density.

```

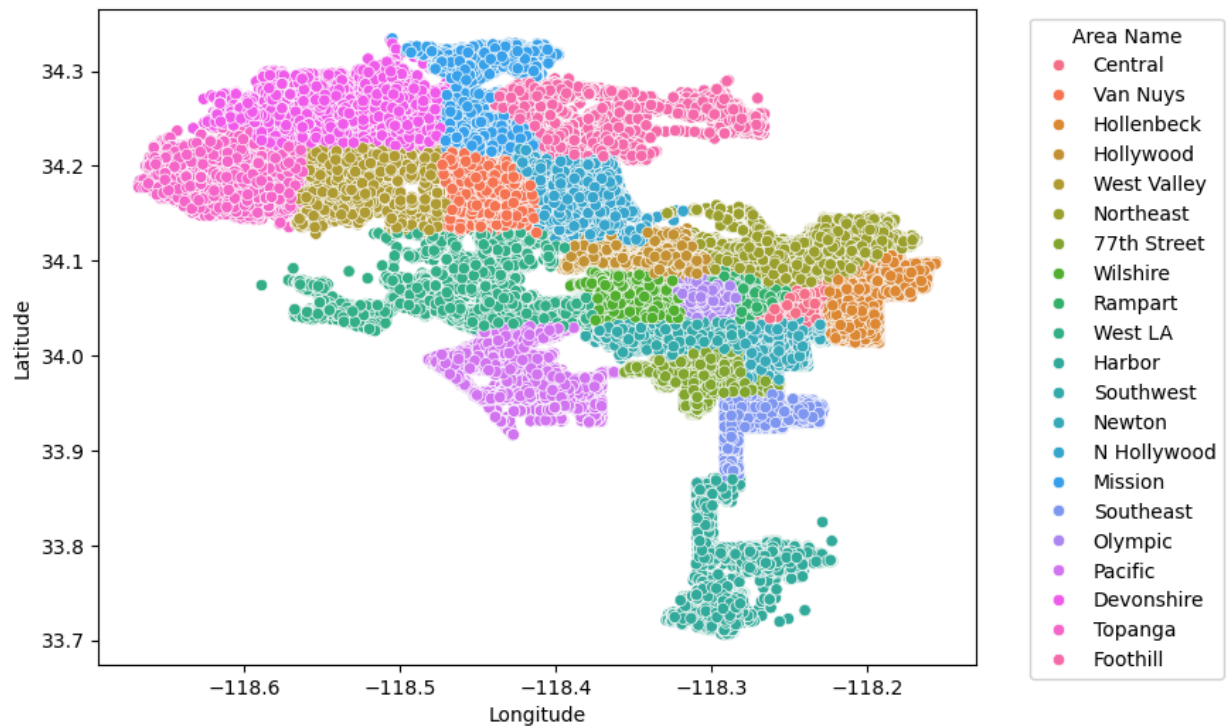
In [10]: df_subset = df[['Latitude', 'Longitude', 'Area_Name']]
df_subset.head()

df_subset = df_subset[(df_subset['Latitude'].between(33, 35)) &
                      (df_subset['Longitude'].between(-119, -117))]

#plot
fig, ax = plt.subplots(figsize=(8,6))
ax = sns.scatterplot(x='Longitude', y='Latitude', hue='Area_Name', data=df_subset, legend='f
plt.legend(title='Area Name', bbox_to_anchor=(1.05, 1), loc='upper left')

plt.show();

```



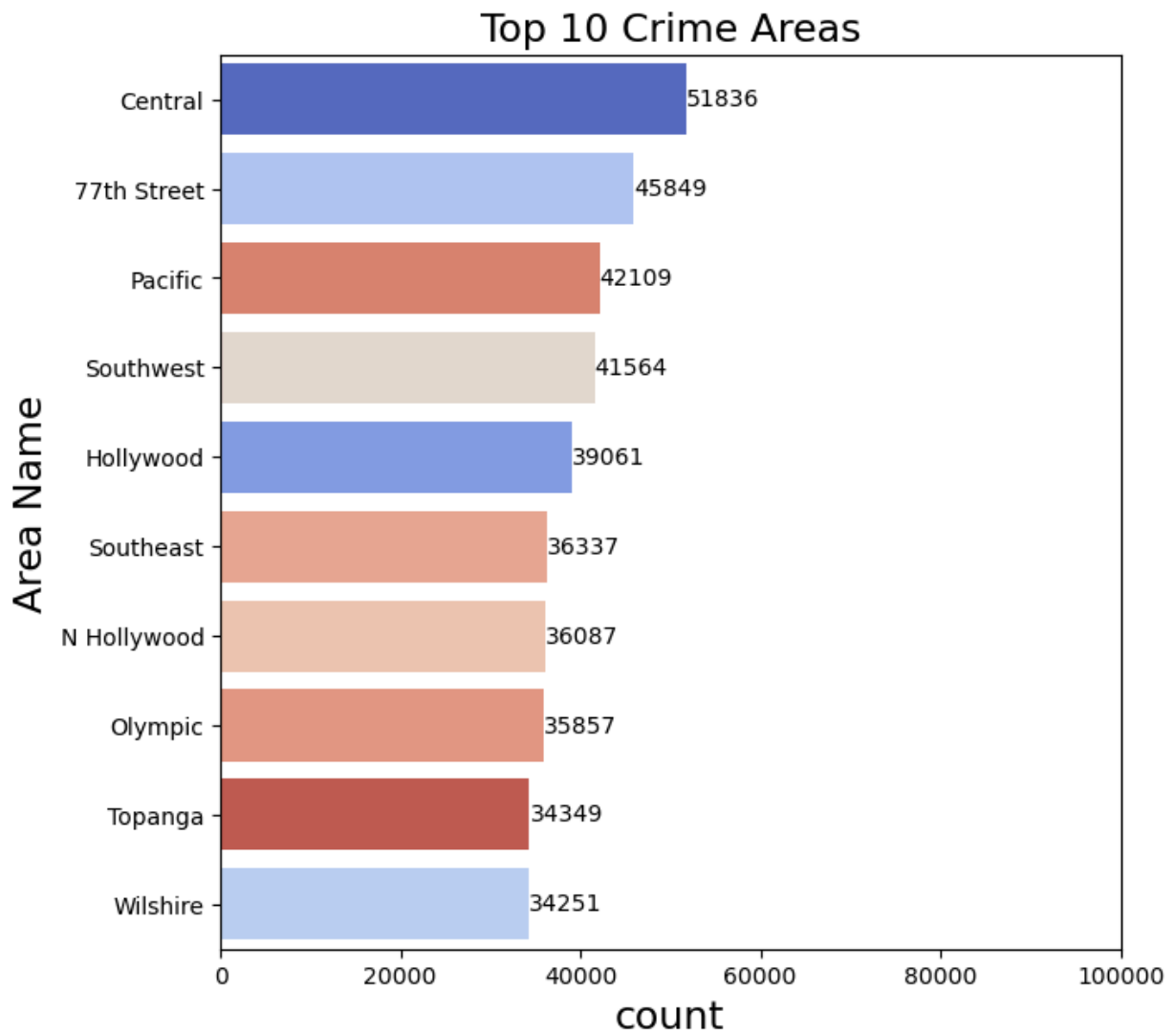
```
In [11]: # Displaying only top 10 crime Areas.
plt.figure(figsize=(7, 7))

top_10_crimes = df['Area_Name'].value_counts().nlargest(10)

crime_plot = sns.countplot(data = df, y='Area_Name', hue='Area_Name', order=top_10_crimes.index)

plt.title('Top 10 Crime Areas', fontsize = 17)
plt.xlabel('count', fontsize = 17)
plt.ylabel('Area Name', fontsize = 17)
plt.xlim(0, 100000)

for bar in crime_plot.patches:
    plt.text(
        bar.get_width(),
        bar.get_y() + bar.get_height() / 2,
        f'{int(bar.get_width())}',
        va='center'
    )
plt.show();
```



Crime Timing Analysis: When Do Crimes Occur?**

The time-based analysis revealed **distinct patterns of criminal activity by hour, day, and season**:

Observation:

Peak Times: The most common times for crimes are

- **12 PM (Noon)** – This is the peak period for criminal activity, likely due to theft and robbery.
- **6 PM and 8 PM** – There is an increase in criminal activity during these hours as people are commuting home.
- **5 PM and 10 PM** – Crimes are often associated with nightlife, coinciding with reduced law enforcement presence.

Seasonal Trends: Crime rates are highest in spring and winter, with an uptick in offenses during these colder months.

Monthly Data: January consistently sees the highest number of crimes, which may be attributed to post-holiday financial strain and theft-related offenses.

```
In [13]: crime_on_Time = df['Victim_Age'].groupby(df['Time_Occurred']).agg(['mean', 'count'])

crime_Time = crime_on_Time.sort_values(by=('count'), ascending = False)[:5]

# Apply background gradient for visualization
crime_Time.style.background_gradient(cmap='Greens')

crime_on_Time = df['Victim_Age'].groupby(df['Time_Occurred']).count()
crime_on_Time
```

```
Out[13]:
```

	mean	count
Time_Occurred		
12:00 PM	41.950878	26241
06:00 PM	40.262545	16481
08:00 PM	38.924124	15947
05:00 PM	41.047679	15877
10:00 PM	38.471992	14460

```
Out[13]: Time_Occurred
01:00 AM      8420
01:00 PM     11540
01:01 AM       78
01:01 PM       55
01:02 AM       44
...
12:57 PM       86
12:58 AM       51
12:58 PM       69
12:59 AM       44
12:59 PM       67
Name: Victim_Age, Length: 1439, dtype: int64
```

```
In [14]: # Without age restriction here's the Time_occurred hourly. To show when crime is the highest
df['Hour'] = pd.to_datetime(df['Time_Occurred'], format='%I:%M %p', errors='coerce').dt.strf
crime_on_Time = df.groupby('Hour')['Victim_Age'].count().sort_values(ascending=False)[:5]
crime_on_Time
```

```
Out[14]: Hour
12 PM      52054
06 PM      40767
05 PM      40413
08 PM      39297
03 PM      38048
Name: Victim_Age, dtype: int64
```

```
In [15]: df_crosstab = pd.DataFrame(pd.crosstab(df['Crime_Description'], df['season']))

# Add a Total column summing all seasons
df_crosstab['Total'] = df_crosstab['Fall'] + df_crosstab['Spring'] + df_crosstab['Summer'] +

# Sort by Total in descending order and keep only the top 10
df_crosstab = df_crosstab.sort_values(by='Total', ascending=False).head(10)

# Display with color formatting
df_crosstab.style.background_gradient(cmap='coolwarm')
```

Out[15]:

	season	Fall	Spring	Summer	Winter	Total
Crime_Description						
BATTERY - SIMPLE ASSAULT		17615	18928	18425	18631	73599
BURGLARY FROM VEHICLE		14975	15113	14382	16801	61271
THEFT OF IDENTITY		14289	15516	14288	17057	61150
ASSAULT WITH DEADLY WEAPON, AGGRAVATED ASSAULT		12511	12904	13419	12181	51015
THEFT PLAIN - PETTY (\$950 & UNDER)		11351	11376	11262	12373	46362
INTIMATE PARTNER - SIMPLE ASSAULT		10910	12029	11555	11725	46219
VANDALISM - FELONY (\$400 & OVER, ALL CHURCH VANDALISMS)		10953	11546	11774	11057	45330
BURGLARY		8973	10172	8978	10667	38790
THEFT FROM MOTOR VEHICLE - GRAND (\$950.01 AND OVER)		8618	8606	8046	9689	34959
THEFT-GRAND (\$950.01 & OVER)EXCPT,GUNS,FOWL,LIVESTK,PROD		6807	6572	6805	7063	27247

```
In [16]: #Crosstab to show the count of crime for each months of the year. From 2020 to 2024. Results
month_order = [
    "January", "February", "March", "April", "May", "June",
    "July", "August", "September", "October", "November", "December"]
Year_month = pd.crosstab(df['Month'],df['Year']).reset_index()
Year_month.index = pd.CategoricalIndex(Year_month['Month'], categories = month_order, ordered = True)
Year_month = Year_month.sort_index().reset_index(drop = True)

# Display with color formatting
Year_month.style.background_gradient(cmap='coolwarm')
```

Out[16]:

	Year	Month	2020	2021	2022	2023	2024
0	January		14626	12331	13691	14785	12720
1	February		13587	11412	13381	13790	11798
2	March		12254	12348	14847	13921	10898
3	April		11525	12141	14914	13417	8507
4	May		12503	12965	15272	13653	5142
5	June		12426	13053	15338	13587	3794
6	July		12752	14412	14998	14381	3757
7	August		12648	13920	15161	14289	3685
8	September		11844	13901	14653	13638	3797
9	October		12262	14427	15397	14185	3833
10	November		11690	13676	13959	13122	3662
11	December		11811	13160	15151	13266	2844

Most Common Crimes

We identify the **most frequent crimes in Los Angeles** and examine their relative occurrences. An analysis of crime descriptions revealed the ten most common crimes:

Observation:

- **Battery - Simple Assault** was the most frequently reported offense.
- **Burglary from Vehicles** and **Theft-Related Offenses** remained significant throughout the year.
- **Aggravated Assault** and **Robbery** consistently ranked among the top violent crimes.

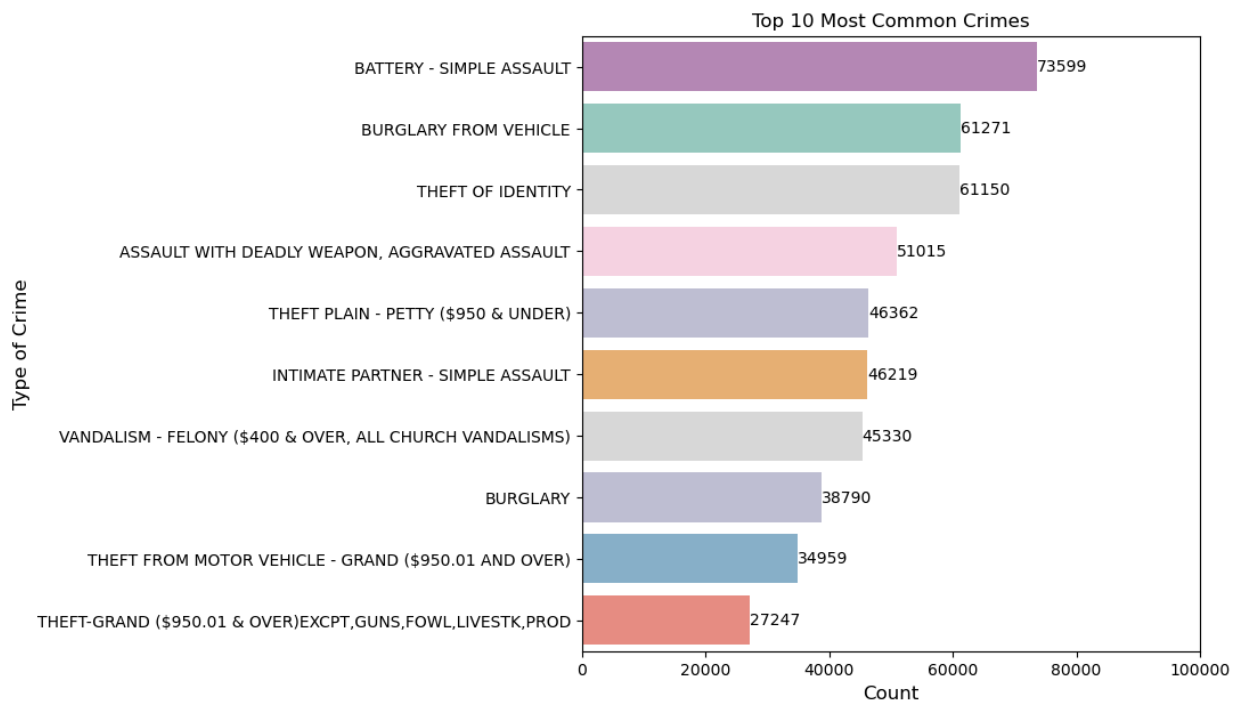
```
In [18]: # Displaying the top 10 most common type of crimes
plt.figure(figsize=(7, 7))

top_crimes = df['Crime_Description'].value_counts().nlargest(10)

crime_plot = sns.countplot(data=df, y = 'Crime_Description', hue = 'Crime_Description', legend=True)

plt.title('Top 10 Most Common Crimes', fontsize = 12)
plt.xlabel('Count', fontsize = 12)
plt.ylabel('Type of Crime', fontsize = 12)
plt.xlim(0, 100000)

for bar in crime_plot.patches:
    plt.text(
        bar.get_width(),
        bar.get_y() + bar.get_height() / 2,
        f'{int(bar.get_width())}',
        va='center'
    )
plt.show();
```



Trends over Time

- A trend analysis over the years showed a peak in **early 2022**, likely influenced by pandemic-related shifts in crime patterns, such as an increase in domestic violence and theft.
- However, a sharp decline was observed in 2024, which is due to **incomplete data** reporting rather than an actual reduction in crime.

```
In [20]: plt.figure(figsize=(10,5))

# Counting occurrences of each Crime Type across all years
df_count = df.groupby('Crime_Description').size().reset_index(name='Total_Count')

# The top 10 crime types based on total occurrences
top_10_crimes = df_count.nlargest(10, 'Total_Count')['Crime_Description']

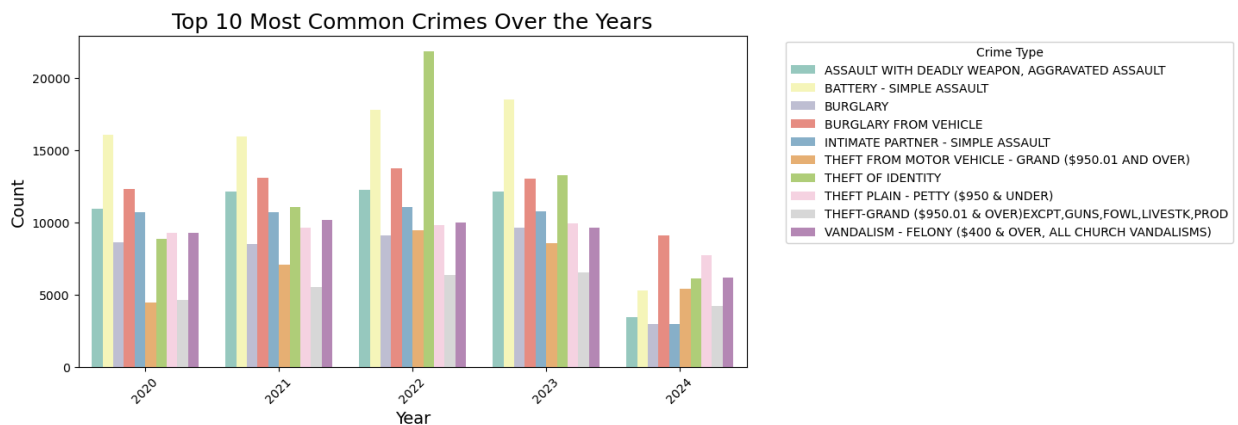
# Filtering to include only these top 10 crime types
df_top10 = df[df['Crime_Description'].isin(top_10_crimes)]

# Group by Year and Crime Type to count occurrences per year
df_top10_grouped = df_top10.groupby(['Year', 'Crime_Description']).size().reset_index(name='Count')

# Plot
sns.barplot(data=df_top10_grouped, x='Year', y='Count', hue='Crime_Description', palette='Set2')

plt.title('Top 10 Most Common Crimes Over the Years', fontsize=18)
plt.xlabel('Year', fontsize=14)
plt.ylabel('Count', fontsize=14)
plt.xticks(rotation=45)
plt.legend(title='Crime Type')

plt.legend(title='Crime Type', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.show();
```



Premise of the Top Crimes

Analysis of crime locations revealed that the **most frequent sites** for criminal activity were **single-family homes, streets, and multi-unit residences**.

Observations:

- Residential crimes tended to increase during the winter months, indicating a rise in burglary incidents around the holiday season.
- Public spaces such as parking lots and sidewalks were identified as hotspots for theft and assaults.

```
In [22]: df_crosstab= pd. DataFrame(pd. crosstab(df[ 'Premises_Description'],df [ 'season']))
# get a total count of reviews per Premises Description
df_crosstab[ 'Total'] = df_crosstab[ 'Fall'] + df_crosstab[ 'Spring'] + df_crosstab[ 'Summer']
df_crosstab = df_crosstab.sort_values(by='Total', ascending=False).head(5)
df_crosstab.style.background_gradient(cmap='Purples')
```

Out[22]:

	season	Fall	Spring	Summer	Winter	Total
Premises_Description						
SINGLE FAMILY DWELLING		37709	41290	38560	42434	159993
STREET		32507	33539	33571	34656	134273
MULTI-UNIT DWELLING (APARTMENT, DUPLEX, ETC)		26631	28760	27462	30151	113004
PARKING LOT		11168	11774	11334	12187	46463
SIDEWALK		9302	9530	9679	9176	37687

Gender & Ethnicity

1. Gender Analysis:

- The pie chart indicates that male victims (51.1%) slightly outnumber female victims (48.9%) in crime incidents.

2. Ethnicity Analysis:

- The Hispanic/Latinx population reported the highest number of crime victims, followed by White and Black individuals.

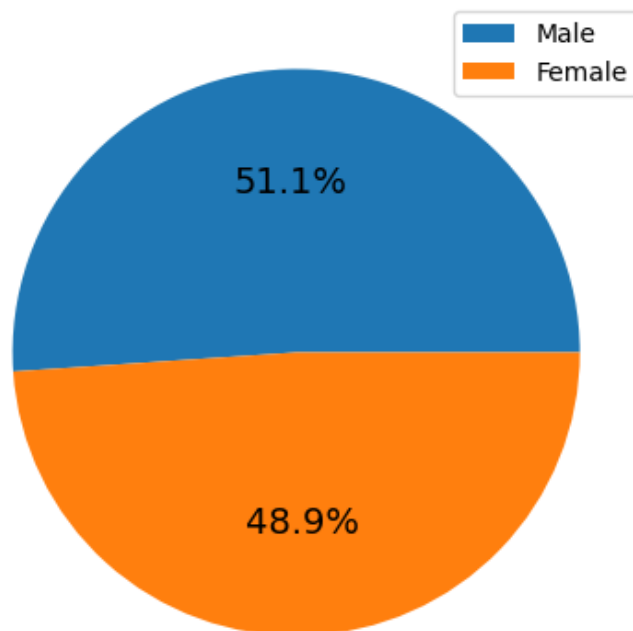
Observations:

- Although the difference is minor, it suggests that both genders are nearly equally affected by crime
- This disparity may be influenced by demographic distribution and socioeconomic factors that affect exposure to crime.
- The age distribution of victims suggests that **young Adults (20-40 years old) are the most affected age group**.

```
In [24]: labels = ['Male', 'Female']
counts = df['Victim_Sex'].value_counts()
counts = counts[counts > 100]

plt.figure(figsize=(5, 5))
plt.title('Percentage of Crime Victims Based on Gender')
plt.pie(counts, autopct='%1.1f%%', textprops={"fontsize":14})
plt.legend(labels, loc="best")
plt.show();
```

Percentage of Crime Victims Based on Gender



```
In [25]: crime_on_Ethnicity = df['Victim_Age'].groupby(df['Victim_Ethnicity']).agg(['mean', 'count'])

crime_Ethnicity = crime_on_Ethnicity.sort_values(by=('count'), ascending = False)[:5]

# Apply background gradient for visualization
crime_Ethnicity.style.background_gradient(cmap='Greens')
```

Out[25]:

	mean	count
--	------	-------

Victim_Ethnicity		
Hispanic/Latin/Mexican	36.910739	291268
White	43.515353	186212
Black	39.455200	134073
Other	41.832969	64491
Asian	42.295707	20987

```
In [26]: crime_on_year = df['Year'].groupby(df['Victim_Age']).agg(['mean', 'count'])

crime_year = crime_on_year.sort_values(by=('count'), ascending = False)[:5]

# Apply background gradient for visualization
crime_year.style.background_gradient(cmap='Greens')
```

Out[26]:

	mean	count
--	------	-------

Victim_Age		
30	2021.805188	22206
35	2021.833456	21772
31	2021.833068	21332
29	2021.811470	21238
28	2021.805364	20880

Summary

This analysis of **Los Angeles crime data from 2020 to 2024** provides valuable insights into **when, where, and what types of crimes occur most frequently**.

The findings reveal that **crime rates peak during spring and winter**, with **noon and evening hours experiencing the highest levels of criminal activity**.

The **areas most affected include Central LA, 77th Street, and Pacific**, where crime density is greatest.

Common crimes such as **assault, burglary, and theft-related offenses** continue to dominate, underscoring the need for effective law enforcement strategies.

Challenges

Underreporting: Some crimes may not be officially reported, resulting in data gaps.

Data Incompleteness in 2024: A noticeable drop in recorded crimes in 2024 likely stems from missing data rather than an actual reduction in crime rates.

Anonymized Geospatial Data: Location data is generalized, which reduces the precision of mapping exact crime locations.

Next Steps

Predictive Modeling: Apply machine learning techniques to forecast crime likelihood based on historical trends.

Granular Time-Series Analysis: Further explore specific crime trends on a monthly basis to detect hidden patterns.

Weapon Usage Analysis: Investigate patterns in firearm-related offenses and their correlation with violent crime rates. crime rates.

In [29]: `df.to_csv("Cleaned_Crime_Data_FinalReport.csv", index=False)`